

Operating Systems (Hinglish)

Top 50 Interview Questions & Detailed Solutions

Simplified Technical Preparation Guide
Hindi + English Mixed for Easy Understanding

1. Core Concepts & Architecture

Q1: What is an Operating System, and what are its primary functions?

Operating System (OS) एक core system software है जो कंप्यूटर के hardware और user applications के बीच एक interface या bridge की तरह काम करता है। इसके primary functions ये हैं:

- **Resource Management:** CPU time, memory space, और raw hardware devices को अलग-अलग running programs में dynamically allocate करना।
- **Process Management:** Threads/processes को schedule करना, create करना, synchronize करना और ज़रूरत पड़ने पर terminate करना।
- **Memory Management:** कौन सा data RAM में कहाँ लोड होगा, उसे track करना और virtual address को physical address में map करना।
- **File System Control:** Disk पर मौजूद raw binary data को structured directories और files के रूप में organize करना।
- **Security and Isolation:** एक program को दूसरे program के memory space में तांक-झांक करने से रोकना ताकि पूरा system crash न हो।

Q2: Explain the difference between Kernel Mode and User Mode. Why is this separation necessary?

System की stability और security बनाए रखने के लिए CPU दो execution modes में काम करता है:

- **User Mode:** इस mode में सारे regular user applications (जैसे Chrome, VS Code) चलते हैं। इनके पास hardware का direct access नहीं होता और न ही ये सीधे system memory को modify कर सकते हैं। अगर यहाँ कोई app crash होता है, तो सिर्फ वही app बंद होता है, पूरा OS नहीं।
- **Kernel Mode (Supervisor Mode):** यह OS के core parts (Kernel) के लिए reserved होता है। यहाँ code के पास hardware, system configuration, और physical RAM का absolute और unrestricted access होता है।

Why it's necessary: यह separation इसलिए ज़रूरी है ताकि कोई भी normal malicious या buggy program सीधे hardware registers को overwrite न कर सके और system को crash होने से बचाया जा सके।

Q3: What is a System Call? Walk through what happens when a program executes a system call.

जब किसी user-space application को कोई privileged काम करना होता है (जैसे file read करना, network पर data भेजना, या नया process बनाना), तो वह सीधे hardware को access नहीं कर सकता। वह Kernel से request करने के लिए **System Call** का इस्तेमाल करता है।

Execution Step-by-Step:

1. User program registers में parameters set करता है और एक trap instruction (जैसे `syscall` या `int 0x80`) execute करता है।
2. CPU तुरंत **User Mode से Kernel Mode** में switch हो जाता है।

3. Hardware, **Interrupt Vector Table / System Call Table** में जाकर उस specific System Call ID के handler function को ढूँढता है।
4. Kernel उस specific function को safe तरीके से execute करता है।
5. काम पूरा होने के बाद, kernel return configurations set करता है और वापस execution को **User Mode** में flip कर देता है।

Q4: Compare a Monolithic Kernel vs. a Microkernel. What are the pros and cons of each?

- **Monolithic Kernel (e.g., Linux, Windows):** इसमें OS की सारी core services (जैसे memory management, scheduler, VFS, device drivers) एक ही बड़े single address space (Kernel space) के अंदर चलती हैं।

Pros: यह बहुत fast होता है क्योंकि component interaction के लिए बार-बार context switch नहीं करना पड़ता।

Cons: अगर एक भी third-party device driver crash हुआ, तो पूरा kernel panic हो जाएगा और system blue-screen/dead हो जाएगा।

- **Microkernel (e.g., QNX, L4):** इसमें kernel को एकदम bare-minimum रखा जाता है (सिर्फ IPC, basic scheduling, basic memory management)। बाकी सारी चीज़ें (File systems, drivers) user space में अलग independent servers की तरह चलती हैं।

Pros: यह highly secure और crash-resilient होता है। अगर network driver crash हुआ, तो kernel बाकी system को बिना प्रभावित किए उसे चुपचाप restart कर देता है।

Cons: components के बीच भारी Inter-Process Communication (IPC) होने की वजह से performance overhead बढ़ जाता है।

Q5: What is Dual-Mode operation in an OS, and how does the hardware support it?

Dual-Mode operation का मतलब है कि system hardware-level पर कम से कम दो privileges को support करता है: User Mode (unprivileged) और Monitor/Kernel Mode (privileged)।

Hardware Support: इसे CPU के अंदर मौजूद एक physical **Mode Bit** (Processor Status Word register में) के ज़रिए enforce किया जाता है। User Mode के लिए यह bit 1 होती है और Kernel Mode के लिए 0। अगर कोई program User Mode (bit 1) में रहते हुए किसी privileged CPU instruction (जैसे hardware interrupts disable करना) को चलाने की कोशिश करता है, तो hardware उसे तुरंत block करके OS को trap signal भेज देता है।

Q6: Explain the concept of Virtualization in the context of operating systems.

Virtualization एक ऐसी abstraction technique है जिसमें एक ही physical hardware controller/assets को **Hypervisor** या **Virtual Machine Monitor (VMM)** की मदद से multiple isolated virtual machines (VMs) में बांट दिया जाता है। Hypervisor physical resource layer को emulate करता है, जिससे हर "Guest OS" को ऐसा लगता है कि उसके पास अपना खुद का standalone dedicated CPU और RAM मौजूद है।

Q7: What is a bootstrapping program (Bootloader), and where is it stored?

जब हम computer का power button दबाते हैं, तो CPU सबसे पहले जिस software को execute करता है उसे **Bootloader** (या bootstrapping program) कहते हैं। इसका काम basic motherboard systems को initialize करना, storage drive को locate करना, और OS kernel image को RAM में load करके execution kernel को सौंपना होता है।

यह motherboard पर लगी एक non-volatile firmware chip में स्टोर होता है, जिसे पुराने systems में **BIOS** और नए modern devices में **UEFI** कहते हैं।

Q8: What is RTOS (Real-Time Operating System)? Distinguish between "Hard" and "Soft" real-time systems.

RTOS एक ऐसा specialized operating system होता है जहाँ processing time-limits (deadlines) और deterministic behavior सबसे बड़ी priority होते हैं, न कि average throughput।

- **Hard RTOS:** यहाँ deadline miss करने का मतलब है—total system failure या physical catastrophe (जैसे: car ABS brakes, missile guidance loops, pacemaker devices)। इसमें late data की value zero होती है।
- **Soft RTOS:** यहाँ deadlines critical तो होती हैं, लेकिन अगर कभी-कभार थोड़ी देर हो भी जाए, तो catastrophic failure नहीं होता, बस user experience/QoS थोड़ा degrade हो जाता है (जैसे: audio/video streaming apps, online gaming setup)।

Q9: What are Interrupts and Exceptions? How does the OS handle an interrupt?

- **Interrupt:** यह एक asynchronous signal होता है जो external hardware devices (जैसे keyboard press या disk ready signal) द्वारा CPU को यह बताने के लिए भेजा जाता है कि उसे तुरंत attention चाहिए।
- **Exception:** यह एक synchronous event होता है जो खुद CPU के अंदर चल रहे code की गलती या validation failure की वजह से generate होता है (जैसे: divide by zero, page fault)।

Handling Mechanism: जैसे ही interrupt आता है, CPU current execute हो रहे instructions को रोकता है, program counter (PC) और processor registers की current state को kernel stack पर save करता है, **Interrupt Vector Table** से handler key (ISR - Interrupt Service Routine) ढूँढता है, उसे run करता है, और काम होने पर पुराने process को वहीं से resume कर देता है।

Q10: What is DMA (Direct Memory Access), and why is it vital for high-performance I/O?

DMA एक special hardware design feature है जिसके ज़रिए high-speed I/O controllers (जैसे high-speed NVMe SSDs, network cards) सीधे system RAM के साथ data blocks read/write कर सकते हैं, और इस पूरे process में CPU को बीच में इन्वॉल्व नहीं होना पड़ता।

Why it's vital: अगर DMA नहीं होगा, तो हर single byte data को transfer करने के लिए CPU को एक loop चलाना पड़ेगा या interrupt handle करना पड़ेगा। इससे network downloading या file copying के दौरान CPU utilization 100% पर अटक जाएगा। DMA की मदद से CPU फ्री रहता है और parallel में application calculations कर सकता है।

2. Process Management & CPU Scheduling

Q11: What is a Process, and how does it differ from a Program?

- **Program:** यह एक passive entity (inactive file) है, जो static machine codes के रूप में आपके secondary hard drive पर पड़ी रहती है (जैसे: app.exe या binary asset)।
- **Process:** यह एक active entity है। जब आप उस program को execute करते हैं, तो वह RAM में लोड हो जाता है। एक process के पास अपनी खुद की memory layout (Text, Data, Heap, Stack), CPU registers context, और open file handles होते हैं।

Q12: Explain the different states of a process (New, Ready, Running, Waiting, Terminated).

एक process अपने lifecycle में इन states से गुज़रता है:

- **New:** Process अभी-अभी create हो रहा है और इसके OS structures initialize हो रहे हैं।
- **Ready:** Process पूरी तरह RAM में loaded है और CPU core मिलने का इंतज़ार कर रहा है।
- **Running:** CPU core पर फिलहाल इसी process के instructions execute हो रहे हैं।
- **Waiting (Blocked):** Process किसी resource, user input, या I/O execution के पूरा होने का इंतज़ार कर रहा है।
- **Terminated:** Process का काम पूरा हो चुका है, और OS इसकी memory resources को clear/reclaim कर रहा है।

Q13: What is a Process Control Block (PCB)? What specific information does it hold?

PCB kernel का एक vital data structure होता है, जो हर active process की पूरी कुंडली (metadata) रखता है ताकि OS उसे ठीक से manage कर सके। इसमें यह information होती है:

- **Process ID (PID):** unique identification index number.
- **Process State:** current state (Ready, Blocked, etc.).
- **Program Counter (PC):** अगले कौन से instruction नंबर को execute करना है उसका track address।
- **CPU Registers:** Context switch के वक्त accumulator, stack pointer, और context properties की saved values।
- **Memory Management limits:** page tables base references.
- **I/O Status:** Opened files और hardware allocation permissions की list।

Q14: What is Context Switching? Why is it considered an overhead?

जब CPU एक process/thread को रोककर (उसकी state save करके) दूसरे process/thread को CPU core सौंपता है, तो इस mechanism को **Context Switching** कहते हैं।

Why it's an overhead: जब context switch हो रहा होता है, तब CPU कोई भी useful computation (काम) नहीं कर रहा होता। पुराना PCB update करना, नए data frames registers में reload करना, page maps बदलना, और CPU caches (L1/L2 cache) + TLB data को invalid/flush करना पड़ता है, जो बहुत heavy performance cost लेता है।

Q15: What is a Thread? Explain the difference between a process and a thread, and when you would use one over the other.

Thread किसी भी process के अंदर का एक **lightweight execution unit** होता है। एक ही process के multiple threads हो सकते हैं।

Differences: Processes डिफॉल्ट रूप से आपस में memory share नहीं करते, जबकि एक ही process के सारे threads आपस में **Text (Code)**, **Data**, और **Heap** segments शेयर करते हैं, लेकिन हर thread का अपना personal **Stack**, **Registers**, और **Program Counter** होता है।

When to use: जब काम के बीच बहुत ज़्यादा communication और data sharing की ज़रूरत हो (जैसे parallel computational math या video rendering tasks), तो **Threads** का यूज़ करते हैं। जब हमें strict boundary और isolation चाहिए कि एक crash दूसरे को प्रभावित न करे (जैसे browser tabs या untrusted plug-in execution), तो **Processes** का यूज़ करते हैं।

Q16: Differentiate between User-level threads and Kernel-level threads.

- **User-level threads:** इन्हें user space की threading libraries (बिना kernel को पता चले) manage करती हैं। ये create और switch होने में सुपर-फास्ट होते हैं क्योंकि कोई system call नहीं लगती। लेकिन नुकसान यह है कि अगर एक thread किसी blocking I/O पर अटक गया, तो kernel को लगेगा पूरा process block हो गया है, जिससे बाकी threads भी रुक जाते हैं।
- **Kernel-level threads:** इन्हें सीधा OS kernel जानता और schedule करता है। अगर एक thread block होता है, तो kernel scheduler उसी process के दूसरे thread को किसी दूसरे execution core पर schedule कर देता है। हालांकि, इन्हें switch करने में system call overhead लगता है।

Q17: What is the difference between a Short-term, Medium-term, and Long-term Scheduler?

- **Long-term Scheduler (Job Scheduler):** यह disk storage से jobs चुनकर RAM की ready queue में लाता है। यह **degree of multiprogramming** (RAM में एक साथ कितने processes रहेंगे) को control करता है। यह काफी कम frequency पर चलता।
- **Short-term Scheduler (CPU Scheduler):** यह ready queue में से अगला process चुनकर उसे सीधे CPU core सौंपता है। यह बहुत fast और millisecond-level पर चलता है।
- **Medium-term Scheduler (Swapper):** जब RAM फुल होने लगती है, तो यह active processes को RAM से उठाकर secondary disk (Swap space) पर डाल देता है (**Swap Out**) और बाद में जगह होने पर वापस लाता है (**Swap In**)।

Q18: Explain the following CPU scheduling algorithms: FCFS, SJF, Round Robin, and Priority Scheduling.

- **FCFS (First-Come, First-Served):** जो process ready queue में पहले आएगा, उसे CPU पहले मिलेगा। यह simple है लेकिन इसमें Convoy Effect की दिक्कत आ सकती है।
- **SJF (Shortest Job First):** जिसका CPU burst time सबसे छोटा होगा, उसे पहले schedule किया जाएगा। यह average waiting time को न्यूनतम (minimize) करने के लिए best है, लेकिन practical life में अगला burst time predict करना मुश्किल होता है।
- **Round Robin (RR):** Time-sharing systems के लिए best है। हर process को एक छोटा fixed time window (Time Quantum \$q\$) मिलता है। Time खत्म होते ही process queue के पीछे चला जाता है और अगले को मौका मिलता है।
- **Priority Scheduling:** हर process को एक priority rank (integer number) दिया जाता है। Highest priority वाले को CPU पहले मिलता है।

Q19: What is the Starvation problem in CPU scheduling, and how does Aging solve it?

- **Starvation:** जब low-priority या बड़े duration वाले processes को CPU इसलिए नहीं मिल पाता क्योंकि system में लगातार high-priority या छोटे tasks आते रहते हैं, तो इस समस्या को Starvation कहते हैं।
- **Solution (Aging):** Aging एक ऐसी technique है जिसमें ready queue में बैठे-बैठे समय बीतने के साथ-साथ low-priority tasks की internal priority score को धीरे-धीरे बढ़ा दिया जाता है। एक समय ऐसा आता है जब उसकी priority सबसे ज़्यादा हो जाती है और उसे CPU मिल जाता है।

Q20: What is Convoy Effect in FCFS scheduling?

Convoy Effect तब होता है जब एक बहुत बड़ा CPU-bound process ready queue में पहले आ जाता है, and उसके पीछे बहुत सारे छोटे-छोटे I/O-bound tasks अटक जाते हैं। उस अकेले बड़े process के चक्कर में पूरे system का average waiting time बहुत बेकार हो जाता है और resource utilization घट जाता है।

Q21: What is a Preemptive vs. Non-preemptive scheduling algorithm?

- **Non-preemptive:** एक बार अगर किसी process को CPU मिल गया, तो वो उसे तब तक नहीं छोड़ेगा जब तक उसका काम पूरा न हो जाए या वो खुद voluntarily I/O के लिए wait state में न चला जाए।
- **Preemptive:** इसमें OS kernel बीच में ही किसी running process को ज़बरदस्ती रोककर (preempt करके) CPU किसी दूसरे high-priority या ready process को दे सकता है (जैसे dynamic time quantum limits के केस में)।

Q22: What is a Multilevel Queue and Multilevel Feedback Queue scheduling?

- **Multilevel Queue:** Ready queue को कई permanent fixed queues में बांटा जाता है (जैसे Interactive queue अलग, Batch queue अलग)। हर queue की अपनी fixed scheduling strategy होती है और processes एक queue से दूसरे queue में shift नहीं हो सकते।
- **Multilevel Feedback Queue (MLFQ):** यह processes को उनके real-time run behavior के हिसाब से अलग-अलग queues के बीच migrate (move) करने की इज़ाज़त देता है। अगर कोई task CPU बहुत ज़्यादा consume कर रहा है, तो उसे lower-priority queue में डाल दिया जाता है। अगर कोई task बार-बार I/O के लिए yield कर रहा है, तो उसे fast responsive priority queue में promote कर दिया जाता है।

3. Process Synchronization & IPC

Q23: What is Inter-Process Communication (IPC)? Explain the two primary models: Shared Memory and Message Passing.

IPC उन mechanisms का group है जो दो या दो से ज़्यादा independent processes को आपस में data share और coordinate करने में मदद करते हैं।

- **Shared Memory Model:** OS दोनों processes के address space में एक common memory block map कर देता। दोनों बिना kernel के दखल के direct hardware level पर read/write कर सकते हैं। यह सबसे तेज़ (Fastest) होता है, लेकिन race conditions से बचने के लिए user level locks लगाने पड़ते हैं।
- **Message Passing Model:** Processes एक-दूसरे को explicit packets/messages भेजते हैं kernel system calls के थ्रू (जैसे Pipes, Sockets, Mails)। यह safely isolated और scalable होता है, लेकिन हर message पर system call cost की वजह से थोड़ा slow होता है।

Q24: What is the Critical Section Problem, and what three requirements must any solution satisfy?

Critical Section code का वह हिस्सा होता है जहाँ कोई shared resource (जैसे global variables, common files) access या update किया जा रहा हो। अगर multiple threads एक साथ इसमें घुस जाएं, तो data corrupt हो सकता है। किसी भी solution को इन 3 requirements को पूरा करना ही होगा:

1. **Mutual Exclusion:** अगर process P_i अपने Critical Section में है, तो कोई भी दूसरा process उस resource के critical section में नहीं आ सकता।
2. **Progress:** If no process is in its critical section and some processes want to enter, dynamically only those tasks not executing in remainder section can choose what runs next.
3. **Bounded Waiting:** एक process द्वारा request करने के बाद और उसे permission मिलने से पहले, दूसरे processes कितनी बार अंदर जा सकते हैं, उसकी एक limit होनी चाहिए, ताकि कोई process हमेशा के लिए भूखा (starve) न रहे।

Q25: Explain the concept of Race Condition with a real-world programming example.

Race Condition तब होती है जब concurrent execution का final output इस बात पर depend करने लगे कि threads किस specific sequence या timing में execute हुए हैं।

Example: मान लेते हैं कि दो threads एक shared variable `counter = 5` को एक साथ increment करना चाहते हैं। Assembly level पर `counter++` 3 steps में होता है:

```
1. REG = counter;      // Load value from RAM to register
2. REG = REG + 1;      // Increment value in register
3. counter = REG;      // Store value back to RAM
```

Thread A आया, उसने register में 5 read किया और उसे 6 कर दिया, लेकिन step 3 से पहले context switch हो गया। अब Thread B आया, उसने भी RAM से 5 read किया, 6 किया और RAM में 6 लिख दिया। जब Thread A वापस आया, उसने अपना पुराना saved register value 6 वापस RAM में write कर दिया। दो बार increment होने के बाद भी final value 6 रह गई, जबकि 7 होनी चाहिए थी।

Q26: What is a Mutex (Mutual Exclusion lock), and how does it differ from a Semaphore?

- **Mutex:** यह एक locking primitive है जिसका उपयोग केवल एक shared resource को exclusive protection देने के लिए किया जाता है। इसमें **Ownership Semantics** होती है—यानी जिस specific thread ने lock को acquire किया है, केवल वही उसे release/unlock कर सकता है।
- **Semaphore:** यह एक signaling primitive है जो एक status value internal integer counter पर काम करता है। इसमें कोई ownership नहीं होती। कोई भी thread या external hardware loop interrupt आकर semaphore को signal/increment कर सकता है।

Q27: Explain the difference between a Binary Semaphore and a Counting Semaphore.

- **Binary Semaphore:** इसकी value केवल 0 या 1 हो सकती है। यह देखने में mutex जैसा लगता है, लेकिन इसमें ownership checking का restriction नहीं होता (कोई भी thread lock open कर सकता है)।

- **Counting Semaphore:** इसकी value एक बड़े range domain में हो सकती है। यह आमतौर पर एक **resource pool** को manage करने के लिए यूज़ होता है (जैसे मान लो system में 5 components हैं, तो value 5 सेट होगी। हर wait() पर value एक कम होगी और 0 होने पर नए threads block हो जाएंगे)।

Q28: What is a Spinlock? When is it preferable to use a spinlock over a traditional mutex?

Spinlock एक ऐसा lock है जिसमें अगर resource busy है, तो thread block होने या सोने (sleep) जाने के बजाय एक tight loop में लगातार condition variable को check करता रहता है (यानी CPU पर "spin" करता रहता है)।

When it is preferable: यह multi-core machines पर तब best माना जाता है जब lock बहुत ही छोटे समय (few instructions execution) के लिए चाहिए हो। ऐसे में thread को sleep मोड में ले जाने और वापस जगाने (context switch overhead) में जितना समय लगेगा, उससे कम समय में thread घूमते-घूमते तुरंत lock पा लेता है।

Q29: Describe the classic Producer-Consumer Problem (Bounded-Buffer Problem) and how semaphores solve it.

इसमें एक fixed size buffer होता है। **Producer** threads data generate करके buffer में डालते हैं और **Consumer** threads data बाहर निकालते हैं।

Semaphore Solution: हम 3 semaphores यूज़ करते हैं: mutex (value 1), empty (value N), और full (value 0).

```
// Producer Code
wait(empty); // Check empty slot है या नहीं
wait(mutex); // Critical section lock
// Add item to buffer
signal(mutex); // Critical section unlock
signal(full); // Full slots count बढ़ाओ

// Consumer Code
wait(full); // Check item available है या नहीं
wait(mutex);
// Remove item from buffer
signal(mutex);
signal(empty); // Empty slots count बढ़ाओ
```

Q30: What is the Dining Philosophers Problem, and what does it illustrate about resource allocation?

5 philosophers एक गोल मेज पर बैठे हैं और सबके बीच में एक-एक chopstick रखी है (कुल 5 chopsticks)। खाने के लिए हर philosopher को अपने left और right दोनों chopsticks उठानी पड़ती हैं।

Illustration: यह problem यह दिखाती है कि बिना सही allocation protocol के अगर सब एक साथ अपनी left chopstick उठा लें, तो कोई भी अपनी right chopstick नहीं उठा पाएगा और पूरा system permanently **Deadlock** में फंस जाएगा।

Q31: What is Priority Inversion, and how does Priority Inheritance mitigate it?

Priority Inversion एक ऐसा scenario है जहाँ एक Low-priority thread ने एक ऐसा lock ले रखा है जिसकी ज़रूरत एक High-priority thread को है। High-priority thread block हो जाता है। तभी एक Medium-priority thread आता है, और चूंकि उसे lock

नहीं चाहिए, वह Low-priority thread को preempt करके खुद CPU पर बैठ जाता है। नतीजा यह होता है कि Medium task की वजह से High-priority task indirectly फंसा रह जाता है।

Solution (Priority Inheritance): जैसे ही High-priority thread उस lock पर block होगा, OS तुरंत उस Low-priority thread की priority को बढ़ाकर temporary रूप से High कर देगा ताकि बीच के Medium threads उसे interrupt न कर सकें, और वह जल्दी से काम खत्म करके lock छोड़ दे।

4. Deadlocks

Q32: What is a Deadlock? What are the four Coffman conditions that must hold simultaneously for a deadlock to occur?

Deadlock processes का एक ऐसा permanent combination state है जहाँ हर process एक resource पकड़ कर बैठा है और किसी दूसरे ऐसे resource का इंतज़ार कर रहा है जो उसी set के किसी दूसरे blocked process ने पकड़ रखा है। इसके लिए चार **Coffman Conditions** का एक साथ होना ज़रूरी है:

1. **Mutual Exclusion:** Resources non-shareable होने चाहिए (एक बार में एक ही यूज़ करे)।
2. **Hold and Wait:** Process एक resource पहले से hold किए हो और नए resource के लिए wait कर रहा हो।
3. **No Preemption:** Resources किसी से ज़बरदस्ती छीने नहीं जा सकते, process खुद ही उसे release करेगा।
4. **Circular Wait:** Processes की एक ऐसी cyclic chain बने जिसमें P_0, P_1 का, P_1, P_2 का, और P_n, P_0 के resource का इंतज़ार कर रहा हो।

Q33: How does Deadlock Prevention differ from Deadlock Avoidance?

- **Deadlock Prevention:** यह एक design-time approach है। इसका मकसद इन चारों Coffman conditions में से किसी भी एक condition को structural level पर तोड़ना है (जैसे नियम बना देना कि कोई भी process "Hold and Wait" नहीं करेगा)। इससे resource utilization बहुत खराब हो जाता है।
- **Deadlock Avoidance:** यह एक runtime dynamic approach है। इसमें OS पहले से जानता है कि हर process को maximum कितने resources लग सकते हैं। जब भी कोई process नया resource mangta है, OS एक simulation चलाकर देखता है कि क्या यह request पूरी करने के बाद system किसी "Unsafe State" में तो नहीं चला जाएगा (e.g., Banker's Algorithm)।

Q34: Explain Banker's Algorithm. What is a "Safe State"?

Banker's Algorithm एक deadlock avoidance algorithm है जो multiple resource types के लिए काम करता है। जब भी कोई process resource मांगता है, algorithm system की Available, Allocated, और Max Need vectors को check करके एक imaginary simulation sequence बनाता है।

Safe State: एक ऐसी allocation state जहाँ OS यह guarantee दे सके कि कम से कम एक ऐसा order execution path मौजूद है जिससे सारे processes अपनी maximum limits मांगने पर भी एक-एक करके बिना deadlock के अपना काम पूरा करके terminate हो सकते हैं।

Q35: How does an OS handle Deadlock Detection and Recovery?

इसमें OS पहले से कोई रोकथाम (prevention/avoidance) नहीं करता, बल्कि deadlock को होने देता है।

- **Detection:** OS समय-समय पर एक background thread चलाता है जो **Resource Allocation Graph (RAG)** में cycles detect करता है (DFS या Tarjan's algorithm की मदद से)।
- **Recovery:** Cycle मिलने पर OS इन तरीकों से recover करता है:
 - Process Termination: या तो सारे deadlocked tasks को kill कर दो या एक-एक करके तब तक kill करो जब तक cycle टूट न जाए।
 - Resource Preemption: किसी process से ज़बरदस्ती resource छीनकर उसे किसी पिछले backup safe checkpoint पर roll-back कर देना।

Q36: What is the Ignorance / Ostrich Algorithm in the context of deadlocks?

Ostrich Algorithm का मतलब है—"दिवकत को देखकर शूतुरमुर्ग की तरह रेत में सिर छुपा लेना और यह सोचना कि कोई दिक्कत ही नहीं है।" यानी deadlock को पूरी तरह ignore कर देना।

Why it's used: Windows और Linux जैसे general operating systems इसी का इस्तेमाल करते हैं। उनका मानना है कि real-world systems में deadlock बहुत ही कम होता है, और उसे हर सेकंड check करने की performance cost बहुत ज़्यादा है। इसलिए अगर कभी साल में एक बार deadlock हो, तो सीधे system reboot कर देना बेहतर trade-off है।

Q37: What is the difference between Deadlock and Livelock?

- **Deadlock:** Processes पूरी तरह **blocked/waiting state** में चले जाते हैं। वे कोई CPU cycle consume नहीं करते, बस एक जगह पत्थर की तरह जम जाते हैं।
- **Livelock:** Processes block नहीं होते; वे लगातार अपनी internal states बदल रहे होते हैं और भारी मात्रा में CPU cycles consume करते हैं, लेकिन active progress **zero** होती है। यह अक्सर तब होता है जब दो threads deadlock से बचने के चक्कर में एक-दूसरे के actions को देखकर लगातार back-off करने लगते हैं।

5. Memory Management

Q38: Explain the difference between Logical (Virtual) Address and Physical Address.

- **Logical (Virtual) Address:** यह वह address होता है जो CPU किसी program को running state में execute करते वक्त generate करता है। Program को ऐसा लगता है कि उसे 0 से लेकर maximum limit तक का एक continuous block मिला हुआ है।
- **Physical Address:** यह आपके real RAM chips का actual physical storage hardware structure location address होता है। Program इसे कभी भी सीधे देख या access नहीं कर सकता।

Q39: What is the function of the MMU (Memory Management Unit)?

MMU CPU के अंदर या उसके पास लगा एक dedicated hardware unit होता है। इसका एकमात्र primary काम है— **CPU द्वारा generate किए गए Logical/Virtual Address को runtime पर Physical Address में translate करना**। इसके लिए यह RAM में मौजूद Page Tables को refer करता है।

Q40: Explain the concept of Paging. What is a Page Table, and where is it located?

Paging: यह एक memory allocation scheme है जो physical memory को non-contiguous (टुकड़ों में) allocate करने की सुविधा देती है। यह virtual address space को blocks में बांट देती है जिन्हें **Pages** कहते हैं, और physical RAM को भी उसी समान size के blocks में बांट देती है जिन्हें **Frames** कहते हैं।

Page Table: यह एक in-memory mapping data structure है जो यह track रखता है कि इस specific process का कौन सा Virtual Page, RAM के किस Physical Frame में बैठा है। यह हर process के लिए अलग होता है और **main RAM (physical memory)** के अंदर स्टोर होता है।

Q41: What is the TLB (Translation Lookaside Buffer), and how does it speed up memory access?

चूंकि Page Table खुद RAM में होता है, इसलिए normal cases में CPU को किसी भी data instruction को access करने के लिए दो बार RAM में जाना पड़ता है। यह बहुत slow होता है।

TLB: यह CPU के अंदर लगी एक extremely fast, hardware-associative cache होती है, जो हाल ही में translate किए गए page table entries को स्टोर रखती है। जब भी CPU कोई address ढूंढता है, वह सबसे पहले TLB में देखता है। अगर **TLB Hit** हो जाता है, तो address translation तुरंत हो जाता है और RAM का पहला चक्कर बच जाता है।

Q42: Contrast Paging with Segmentation. Can they be combined?

- **Paging:** इसमें memory को fixed-size system driven blocks (pages) में बांटा जाता है। यह user/programmer को दिखाई नहीं देता। इसमें external fragmentation नहीं होता।
- **Segmentation:** इसमें memory को programmer के logical framework के हिसाब से variable sizes में बांटा जाता है (जैसे: Code section अलग, Stack अलग)। इसमें external fragmentation हो सकता है।

Combination: हाँ, modern operating systems **Paged Segmentation** यूज़ करते हैं। इसमें logical architecture segments में बंटा होता है, लेकिन हर segment को internally uniform management के लिए tiny pages में तोड़ दिया जाता है।

Q43: What is Virtual Memory, and how does it allow programs larger than physical RAM to execute?

Virtual Memory एक ऐसी abstraction technique है जो आपके logical execution limits को physical RAM की storage limitation से आज़ाद कर देती है। यह secondary disk storage (जैसे SSD swap files) को RAM के extension की तरह यूज़ करती है। OS किसी भी process के केवल उन्हीं pages को RAM में लोड करता है जिनकी तुरंत ज़रूरत है (Demand Paging)। बाकी का पूरा program disk पर पड़ा रहता है।

Q44: What is a Page Fault? Step through how the OS handles a page fault.

जब CPU किसी ऐसे virtual page को access करने की कोशिश करता है जो valid तो है, लेकिन फिलहाल physical RAM में loaded नहीं है, तो MMU एक hardware interrupt raise करता है जिसे **Page Fault** कहते हैं।

Handling Steps:

1. MMU OS kernel को trap/interrupt भेजता है।
2. Kernel internal structures check करके verify करता है कि address request valid है या illegal access है।
3. Valid होने पर, OS disk controller को command देकर secondary storage से उस missing page को ढूंढता है।

4. OS RAM में एक खाली physical frame ढूंढता है। अगर RAM फुल है, तो page replacement algorithm चलाकर किसी old page को evict करता है।
5. Disk से page को लाकर RAM के उस frame में write किया जाता है।
6. Process का Page Table update किया जाता है और valid bit को 1 कर दिया जाता है।
7. CPU को instruction वापस restart करने को कहा जाता है।

Q45: Explain the following page replacement algorithms: FIFO, LRU, and Optimal Page Replacement. What is Belady's Anomaly?

जब RAM full हो और नया page लाना हो, तो पुराने page को हटाने के लिए ये algorithms यूज़ होते हैं:

- **FIFO (First-In, First-Out):** जो page memory में सबसे पहले आया था, उसे सबसे पहले बाहर निकाला जाता है।
- **LRU (Least Recently Used):** यह उस page को बाहर निकालता है जिसका इस्तेमाल पिछले सबसे लंबे समय से नहीं हुआ है। यह best practical strategy है।
- **Optimal (OPT):** यह उस page को हटाता है जिसकी ज़रूरत भविष्य में सबसे लंबे समय तक नहीं होने वाली। यह theoretically best है, लेकिन implement करना असंभव है।

Belady's Anomaly: यह FIFO algorithm में दिखने वाली एक अजीब दिक्कत है, जहाँ सामान्यतः RAM frames बढ़ाने पर page faults घटने चाहिए, लेकिन FIFO के कुछ reference strings में **RAM space (frames) बढ़ाने पर भी page faults की संख्या बढ़ जाती है।**

Q46: What is Thrashing, and how can an OS prevent or fix it?

जब system में बहुत सारे processes एक साथ चल रहे हों और RAM कम पड़ जाए, तो हर process को लगातार page faults होने लगते हैं। नतीजा यह होता है कि OS अपना 99% समय सिर्फ disk से page लाने और ले जाने (**swapping**) में ही बर्बाद कर देता है और actual processing progress लगभग zero हो जाती है। इस भयानक state को **Thrashing** कहते हैं।

Prevention: OS **Working Set Model** या Page Fault Frequency monitoring का यूज़ करता है। अगर thrashing detect होती है, तो medium-term scheduler तुरंत कुछ processes को पूरी तरह RAM से बाहर Swap Out फेंक देता है।

6. Storage & File Systems

Q47: What is a File Allocation Table (FAT) vs. an Inode structure in Unix-like systems?

- **FAT (File Allocation Table):** यह एक static layout entry है जिसमें पूरी disk के blocks का एक single static matrix link array होता है। File का directory head entry बस उसके पहले block index को point करता है, और FAT table में हर index अपने अगले block number का pointer रखता है (एक linked list की तरह)।
- **Inode (Index Node):** Unix-like architectures में हर file का अपना एक personal self-contained structural header dictionary object होता है जिसे Inode कहते हैं। यह file का metadata (permissions, owner, size) रखता है और साथ ही direct, single-indirect, और double-indirect pointers का array रखता है। इससे random access में बहुत speed मिलती है।

Q48: Explain the different file allocation methods: Contiguous, Linked, and Indexed Allocation.

- **Contiguous Allocation:** इसमें हर file को disk पर एक continuous linear list of blocks मिलता है। यह random access के लिए बहुत तेज़ है, लेकिन इसमें external fragmentation बहुत भयंकर होता है।
- **Linked Allocation:** इसमें file के blocks disk पर कहीं भी बिखरे हो सकते हैं। हर block के अंदर ही अगले block का hard memory pointer छिपा होता है। इसमें fragmentation नहीं होता, लेकिन random access बहुत slow हो जाता है।
- **Indexed Allocation:** इसमें हर file का अपना एक dedicated **Index Block** होता है, जिसमें उस file के सारे scattered blocks के pointers एक जगह लिखे होते हैं। यह fast random access देता है।

Q49: What is Internal Fragmentation vs. External Fragmentation? How does Compaction address the latter?

- **Internal Fragmentation:** जब किसी process/file को उसकी ज़रूरत से थोड़ा बड़ा fixed-size slot दे दिया जाता है, तो उस block के अंदर की जो खाली बची हुई जगह (wasted space) होती है, उसे Internal Fragmentation कहते हैं।
- **External Fragmentation:** जब dynamic partition allocation की वजह से memory में बहुत सारे छोटे-छोटे free spaces अलग-अलग जगह बिखर जाते हैं। कुल मिलाकर हमारे पास पर्याप्त space होता है, लेकिन वह continuous configuration में नहीं होता।

Compaction: यह external fragmentation को ठीक करने का एक तरीका है। इसमें OS चल रहे processes को memory में shift करके एक तरफ खिसका देता है, जिससे सारे छोटे-छोटे खाली टुकड़े मिलकर एक बड़ा continuous free memory space बन जाते हैं।

Q50: What is RAID? Briefly explain the differences between RAID 0, RAID 1, and RAID 5.

RAID (Redundant Array of Independent Disks) एक ऐसी technology है जिसमें multiple physical hard drives को मिलाकर एक single logical storage layout unit बना दिया जाता है, ताकि data redundancy (safety) या speed को बढ़ाया जा सके।

- **RAID 0 (Striping):** Data blocks को अलग-अलग disks में बांटकर parallel में write किया जाता है। यह speed के लिए बहुत अच्छा है, लेकिन इसमें zero fault tolerance है।
- **RAID 1 (Mirroring):** जो भी data एक drive पर लिखा जाता है, उसकी exact carbon copy (mirror image) दूसरी drive पर भी लिखी जाती है। यह reliability के लिए safe है, लेकिन cost double हो जाती है।
- **RAID 5 (Striping with Distributed Parity):** इसमें data blocks के साथ-साथ एक mathematical check data (Parity) को disks पर distributed तरीके से stripe किया जाता है। यह performance और reliability का बहुत अच्छा mix है।